

NOMBRE: ALEJANDRO JOSE GOMEZ HERNANDEZ

CARNE: 20347

FECHA: 29/09/2024

Comentario General: El objetivo de esta práctica es que exploren la arquitectura transformer que ya debieron haber leído en el paper original.

Cuando lleguen a la parte de entrenamiento y evaluación, procuren obtener el mejor resultado para WER. Cambien parametros, congelen capas convolucionales, etc. Dejen evidencia de todos sus intentos.

Responda las siguientes preguntas después de haber leído el paper original de Wav2Vec2(Se encuentra en: Materiales del profesor/papers/transformers y LLM): ¿Qué es Wav2Vec2 y qué problema intenta resolver? ¿Cómo se diferencia Wav2Vec2 de los métodos tradicionales de reconocimiento de voz? ¿Cuáles son los componentes principales de la arquitectura de Wav2Vec2? ¿Puedes explicar el proceso de entrenamiento de Wav2Vec2? ¿Cómo funciona el aprendizaje auto-supervisado en este contexto? ¿Qué papel juega la cuantización de características de audio en Wav2Vec2? ¿Cuáles son las ventajas y desventajas de usar Wav2Vec2 en comparación con otros modelos de reconocimiento de voz?

1. ¿Qué es wav2vec2?

Es un modelo autosupervisado para aprender representaciones de audio directamente a partir de datos sin etiquetar, lo que reduce la necesidad de grandes cantidades de datos etiquetados para tareas de reconocimiento de voz. Su arquitectura se basa en convoluciones, transformadores y cuantización de representaciones de audio latentes.

2. ¿Qué resuelve wav2vec2?

wav2vec2 resuelve la necesidad de grandes cantidades de datos etiquetados para entrenar sistemas de reconocimiento de voz. Entrena un modelo preciso con muy pocos datos etiquetados y cantidades masivas de datos no etiquetados.

3. Diferencias con otros enfoques

Wav2vec2 a diferencia de otros, este los combina en un solo proceso de entrenamiento. Resuelve una tarea contrastiva donde el modelo debe identificar la representación correcta dentro de un conjunto de distracciones.

4. Arquitectura

wav2vec2 tiene tres componentes principales:

- Codificador de características: Una red de convolución profunda que toma como entrada la señal de audio cruda y produce representaciones latentes.
- Transformador: Se utiliza para generar representaciones contextuales que capturan la información de toda la secuencia de audio.
- Módulo de cuantización: Se utiliza para convertir las representaciones continuas en representaciones discretas mediante la técnica de product quantization y la técnica Gumbel Softmax para obtener una salida diferenciable.

5. Entrenamiento autosupervisado

El modelo se entrena en un esquema autosupervisado, es decir, sin necesidad de datos etiquetados. Durante el pre-entrenamiento, el modelo enmascara segmentos del audio y debe predecir las representaciones correctas para esos segmentos enmascarados. Esto es similar al entrenamiento de modelos de lenguaje como BERT en el procesamiento de lenguaje natural. Posteriormente, se puede ajustar con datos etiquetados para tareas específicas como el reconocimiento de voz.

6. Cuantización de las características de audio

El modelo incluye una etapa de cuantización de las representaciones latentes que permite convertir las características continuas en discretas. Esta cuantización ayuda a que el modelo aprenda representaciones más robustas y generalizables. El uso de múltiples "codebooks" o grupos de cuantización permite la combinación de diferentes características para obtener representaciones ricas del audio.

```
1 #Librerias a instalar, en caso de no estar instaladas
2
3 !pip install datasets
4 !pip install transformers
5 !pip install jiwer
```

```
Requirement already satisfied: yarl<2.0,>=1.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (1.11.1)
Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->datasets) (4.0.3)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub>=0.22.0->datasets) (4.12.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.32.2->datasets) (3.10)
```

Recursos × ...

No te has suscrito.
[Más información](#)

En estos momentos, no tienes ninguna unidad de computación disponible. Los recursos que se ofrecen sin coste económico no están garantizados. Puedes comprar más unidades [aquí](#).

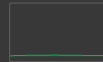
Con tu nivel de uso actual, puede que este entorno de ejecución dure hasta 84 horas.

[Gestionar sesiones](#)

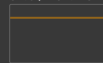
del backend de Google Compute Engine que utiliza Python 3

Mostrando recursos desde las 23:40 a las 23:42

RAM del sistema
1.3 / 12.7 GB



Disco
82.6 / 107.7 GB



```
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.10.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.24.7)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (24.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2024.9.11)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.5)
Requirement already satisfied: tokenizers>=0.20,<=0.19 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.19.1)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.5)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.23.2->transformers) (2024.6.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.23.2->transformers) (4.6.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2024.8.30)
Collecting jiwer
  Downloading jiwer-3.0.4-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: click<9.0.0,>=8.1.3 in /usr/local/lib/python3.10/dist-packages (from jiwer) (8.1.7)
Collecting rapidfuzz<4,>=3 (from jiwer)
  Downloading rapidfuzz-3.10.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)
  Downloading jiwer-3.0.4-py3-none-any.whl (21 kB)
  Downloading rapidfuzz-3.10.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.1 MB)
    3.1/3.1 MB 26.1 MB/s eta 0:00:00
Installing collected packages: rapidfuzz, jiwer
Successfully installed jiwer-3.0.4 rapidfuzz-3.10.0
```

```
1 from datasets import load_dataset
2
3 # Load the TIMIT ASR dataset
4 timit = load_dataset("timit_asr")
5
6 # Display basic information about the dataset
7 print(timit)
8
```

 The repository for timit_asr contains custom code which must be executed to correctly load the dataset. You can inspect the repository content at https://hf.co/datasets/ldc-upenn/timit_asr. You can avoid this prompt in future by passing the argument 'trust_remote_code=True'.

Do you wish to run the custom code? [y/N] y

ManualDownloadError Traceback (most recent call last)

<ipython-input-4-ff788122c6b4> in <cell line: 4>():

```
2
3 # Load the TIMIT ASR dataset
--> 4 timit = load_dataset("timit_asr")
5
6 # Display basic information about the dataset
```

↳ 2 frames

/usr/local/lib/python3.10/dist-packages/datasets/builder.py in _check_manual_download(self, dl_manager)

```
945 def _check_manual_download(self, dl_manager):
946     if self.manual_download_instructions is not None and dl_manager.manual_dir is None:
--> 947         raise ManualDownloadError(
948             textwrap.dedent(
949                 """\
```

ManualDownloadError: The dataset timit_asr with config clean requires manual data.

Please follow the manual download instructions:

To use TIMIT you have to download it manually. Please create an account and download the dataset from

<https://catalog.ldc.upenn.edu/LDC9351>

Then extract all files in one folder and load the dataset with: 'datasets.load_dataset('timit_asr', data_dir='path/to/folder/folder_name')'

Manual data can be loaded with:

datasets.load_dataset("timit_asr", data_dir="path/to/manual/data")

No funcionó el dataset :(

Usaré uno alternativo :D

```
1 from datasets import load_dataset
2
3 # Load the Common Voice dataset for English
4 common_voice = load_dataset("mozilla-foundation/common_voice_11_0", "en")
5
6
7
```

 The repository for mozilla-foundation/common_voice_11_0 contains custom code which must be executed to correctly load the dataset. You can inspect the repository page for more information. You can avoid this prompt in future by passing the argument `trust\_remote\_code=True`.

Do you wish to run the custom code? [y/N] y

```
n_shards.json: 100%  12.2k/12.2k [00:00<00:00, 377kB/s]
Downloading data: 100%  24/24 [05:59<00:00, 17.54s/files]
en_train_0.tar: 100%  1.85G/1.85G [00:20<00:00, 150MB/s]
en_train_1.tar: 100%  1.80G/1.80G [00:12<00:00, 182MB/s]
en_train_2.tar: 100%  1.77G/1.77G [00:10<00:00, 165MB/s]
en_train_3.tar: 100%  1.74G/1.74G [00:11<00:00, 144MB/s]
en_train_4.tar: 100%  1.73G/1.73G [00:12<00:00, 169MB/s]
en_train_5.tar: 100%  1.71G/1.71G [00:14<00:00, 149MB/s]
en_train_6.tar: 100%  1.69G/1.69G [00:10<00:00, 180MB/s]
en_train_7.tar: 100%  1.68G/1.68G [00:16<00:00, 194MB/s]
en_train_8.tar: 100%  1.63G/1.63G [00:09<00:00, 153MB/s]
en_train_9.tar: 100%  1.66G/1.66G [00:13<00:00, 152MB/s]
en_train_10.tar: 100%  1.64G/1.64G [00:12<00:00, 153MB/s]
en_train_11.tar: 100%  1.60G/1.60G [00:13<00:00, 159MB/s]
en_train_12.tar: 100%  1.60G/1.60G [00:10<00:00, 170MB/s]
en_train_13.tar: 100%  1.63G/1.63G [00:12<00:00, 38.2MB/s]
en_train_14.tar: 100%  1.60G/1.60G [00:18<00:00, 155MB/s]
en_train_15.tar: 100%  1.58G/1.58G [00:12<00:00, 25.7MB/s]
en_train_16.tar: 100%  1.67G/1.67G [00:22<00:00, 28.6MB/s]
en_train_17.tar: 100%  1.58G/1.58G [00:15<00:00, 158MB/s]
en_train_18.tar: 100%  1.58G/1.58G [00:21<00:00, 36.6MB/s]
en_train_19.tar: 100%  1.63G/1.63G [00:14<00:00, 28.3MB/s]
en_train_20.tar: 100%  1.48G/1.48G [00:13<00:00, 207MB/s]
en_train_21.tar: 100%  1.76G/1.76G [00:24<00:00, 158MB/s]
en_train_22.tar: 100%  1.89G/1.89G [00:21<00:00, 45.1MB/s]
en_train_23.tar: 100%  1.41G/1.41G [00:13<00:00, 204MB/s]
en_dev_0.tar: 100%  732M/732M [00:10<00:00, 168MB/s]
en_test_0.tar: 100%  722M/722M [00:03<00:00, 202MB/s]
en_other_0.tar: 100%  1.32G/1.32G [00:12<00:00, 174MB/s]
en_other_1.tar: 100%  1.30G/1.30G [00:09<00:00, 170MB/s]
en_other_2.tar: 100%  1.37G/1.37G [00:20<00:00, 157MB/s]
en_other_3.tar: 100%  1.28G/1.28G [00:18<00:00, 184MB/s]
en_other_4.tar: 100%  1.42G/1.42G [00:13<00:00, 154MB/s]
en_other_5.tar: 100%  1.32G/1.32G [00:20<00:00, 44.6MB/s]
en_other_6.tar: 100%  1.39G/1.39G [00:21<00:00, 147MB/s]
en_other_7.tar: 100%  352M/352M [00:08<00:00, 153MB/s]
en_invalidated_0.tar: 100%  1.87G/1.87G [00:27<00:00, 34.1MB/s]
en_invalidated_1.tar: 100%  1.62G/1.62G [00:16<00:00, 151MB/s]
en_invalidated_2.tar: 100%  2.10G/2.10G [00:33<00:00, 142MB/s]
en_invalidated_3.tar: 100%  2.11G/2.11G [00:20<00:00, 148MB/s]
en_invalidated_4.tar: 100%  1.95G/1.95G [00:36<00:00, 124MB/s]
en_invalidated_5.tar: 100%  1.65G/1.65G [00:16<00:00, 144MB/s]
en_invalidated_6.tar: 100%  460M/460M [00:06<00:00, 140MB/s]
```

```
-----
OSError                                Traceback (most recent call last)
<ipython-input-6-8d22cccf749d> in <cell line: 4>()
      2
      3 # Load the Common Voice dataset for English
----> 4 common_voice = load_dataset("mozilla-foundation/common_voice_11_0", "en")
      5
      6

-----
21 frames
/usr/lib/python3.10/tarfile.py in copyfileobj(src, dst, length, exception, bufsize)
    253     if len(buf) < bufsize:
    254         raise exception("unexpected end of data")
--> 255     dst.write(buf)
    256
    257     if remainder != 0:

OSError: [Errno 28] No space left on device
```

```
1 from datasets import load_dataset
2
3 # Cargar la versión pequeña del dataset de Librispeech (clean subset)
4 librispeech = load_dataset("librispeech_asr", "clean", split="validation[:1%]") # Usa solo el 1% para que sea muy pequeño
5
6 # Ver las columnas originales
7 print("Columnas originales:", librispeech.column_names)
8
9
```

```
⚙ /usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Cola
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

librispeech_asr.py: 100% ██████████ 11.4k/11.4k [00:00<00:00, 134kB/s]

README.md: 100% ██████████ 10.2k/10.2k [00:00<00:00, 223kB/s]
The repository for librispeech_asr contains custom code which must be executed to correctly load the dataset. You can inspect the repository content at https
You can avoid this prompt in future by passing the argument 'trust_remote_code=True'.

Do you wish to run the custom code? [y/N] y

Downloading data: 100% ██████████ 338M/338M [00:10<00:00, 39.7MB/s]

Downloading data: 100% ██████████ 347M/347M [00:09<00:00, 40.9MB/s]

Downloading data: 100% ██████████ 6.39G/6.39G [02:50<00:00, 36.7MB/s]

Downloading data: 49% ██████████ 11.3G/23.0G [08:25<05:55, 33.2MB/s]

-----
TimeoutError                               Traceback (most recent call last)
/usr/local/lib/python3.10/dist-packages/fsspec/asyn.py in _runner(event, coro, result, timeout)
    55     try:
--> 56         result[0] = await coro
    57     except Exception as ex:

-----
⚡ 22 frames -----

TimeoutError:

The above exception was the direct cause of the following exception:

FSTimeoutError                               Traceback (most recent call last)
/usr/local/lib/python3.10/dist-packages/fsspec/asyn.py in sync(loop, func, timeout, *args, **kwargs)
    99     if isinstance(return_result, asyncio.TimeoutError):
--> 100         # suppress asyncio.TimeoutError, raise FSTimeoutError
--> 101         raise FSTimeoutError from return_result
    102     elif isinstance(return_result, BaseException):
    103         raise return_result

FSTimeoutError:
```

Pasos siguientes: [Explicar error](#)

```
1 # Load the superb asr_dummy dataset, which is very small
2 from datasets import load_dataset
3
4 # Load the tiny superb ASR dummy dataset for testing
5 dataset = load_dataset("superb", "asr")
6
7 # Print the dataset size and a sample
8 print(f"Dataset size: {len(dataset['train'])} examples")
9 print(dataset['train'][0])
10
```

```
⚙ Generating train split: 100% ██████████ 28539/28539 [00:04<00:00, 10571.92 examples/s]

Generating validation split: 100% ██████████ 2703/2703 [00:00<00:00, 7521.19 examples/s]

Generating test split: 100% ██████████ 2620/2620 [00:00<00:00, 6711.04 examples/s]

Dataset size: 28539 examples
{'file': '/root/.cache/huggingface/datasets/downloads/extracted/39bad2659671b430f631158d09f2c89e593891e6d95eed4dbdda430301560fc0/LibriSpeech/train-clean-100/0.00045776, -0.00946045)', 'sampling_rate': 16000}, 'text': 'CHAPTER ONE MISSUS RACHEL LYNDE IS SURPRISED MISSUS RACHEL LYNDE LIVED JUST WHERE THE AV
```

```
1 # Mantener solo las columnas necesarias: 'text', 'audio', 'id'
2 columns_to_keep = ["text", "audio", "id"]
3
4 # Aplicar la eliminación de columnas en cada subconjunto (train, validation, test)
5 dataset_filtro_train = dataset["train"].remove_columns([col for col in dataset["train"].column_names if col not in columns_to_keep])
6 dataset_filtro_validation = dataset["validation"].remove_columns([col for col in dataset["validation"].column_names if col not in columns_to_keep])
7 dataset_filtro_test = dataset["test"].remove_columns([col for col in dataset["test"].column_names if col not in columns_to_keep])
8
9 # Verificar las columnas filtradas
10 print("Columnas filtradas en el conjunto train:", dataset_filtro_train.column_names)
11 print("Columnas filtradas en el conjunto validation:", dataset_filtro_validation.column_names)
12 print("Columnas filtradas en el conjunto test:", dataset_filtro_test.column_names)
13
14 # Mostrar un ejemplo de los datos en el conjunto de entrenamiento
15 print("Ejemplo del conjunto de entrenamiento:", dataset_filtro_train[0])
16
```

```
⚙ Columnas filtradas en el conjunto train: ['audio', 'text', 'id']
Columnas filtradas en el conjunto validation: ['audio', 'text', 'id']
Columnas filtradas en el conjunto test: ['audio', 'text', 'id']
Ejemplo del conjunto de entrenamiento: {'audio': {'path': '/root/.cache/huggingface/datasets/downloads/extracted/39bad2659671b430f631158d09f2c89e593891e6d95e0.00045776, -0.00946045)', 'sampling_rate': 16000}, 'text': 'CHAPTER ONE MISSUS RACHEL LYNDE IS SURPRISED MISSUS RACHEL LYNDE LIVED JUST WHERE THE AV
```

```
1 from datasets import ClassLabel
2 import random
3 import pandas as pd
4
5 def muestra_textos(dataset, n):
6     n = min(n, len(dataset))
7     sample_indices = random.sample(range(len(dataset)), n)
8     data = [{"id": dataset[i]["id"], "text": dataset[i]["text"]} for i in sample_indices]
9     df = pd.DataFrame(data)
10    return df
```

```
1 import re
2
3 #Limpie los textos.
```

```

4 def limpia_texto(texto):
5     texto = texto.lower()
6     texto = re.sub(r'http\S+|www\S+|https\S+', '', texto, flags=re.MULTILINE)
7     texto = re.sub(r'\@w+', '', texto)
8     texto = re.sub(r'\^w\s]', '', texto)
9     texto = re.sub(r'\d+', '', texto)
10    texto = re.sub(r'\s+', ' ', texto).strip()
11    return texto
12
13

```

```

1 #Define una funcion para construir el vocabulario. Esto es todos los caracteres que están en nuestro corpus.
2 #Ejemplo: {a: 3, b:5, c:7}
3 #Luego, cambie el espacio " " por "|". Esto es una buena practica, para tener un elemento visible para el espacio.
4 #Al vocabulario debe agregar "[UNK]" y "[PAD]". Uno es para desconocidos, otro es para en blanco. Estos deben tener asignada
5 #la longitud del vocabulario.
6 #Guarde el vocabulario en un json: vocab.json
7
8
9 def construir_vocabulario(dataset):
10     """
11     Función para construir el vocabulario de un corpus, reemplazar los espacios con '|',
12     y agregar los tokens '[UNK]' y '[PAD]'.
13     """
14     # Inicializar un contador de caracteres
15     vocab_counter = Counter()
16
17     # Recorrer los textos del dataset para contar los caracteres
18     for example in dataset:
19         texto = example['text'].replace(" ", "|") # Reemplazar los espacios por '|'
20         vocab_counter.update(texto) # Contar los caracteres
21
22     # Ordenar el vocabulario por orden de aparición
23     vocabulario = {char: i for i, (char, _) in enumerate(vocab_counter.most_common())}
24
25     # Agregar [UNK] y [PAD] con la longitud actual del vocabulario
26     vocabulario["[UNK]"] = len(vocabulario)
27     vocabulario["[PAD]"] = len(vocabulario)
28
29     # Guardar el vocabulario en un archivo JSON
30     with open('vocab.json', 'w') as vocab_file:
31         json.dump(vocabulario, vocab_file, ensure_ascii=False, indent=4)
32
33     print(f"Vocabulario creado y guardado como vocab.json. Total de caracteres: {len(vocabulario)}")
34     return vocabulario

```

```

1 from collections import Counter
2 import json
3
4 from transformers import Wav2Vec2CTCTokenizer
5 #Use el vocabulario anterior para tokenizar con Wav2Vec2CTCTokenizer
6 # Construir vocabulario y guardar en vocab.json
7 vocabulario = construir_vocabulario(dataset_filtro_train)
8
9 # Cargar el vocabulario en Wav2Vec2CTCTokenizer
10 from transformers import Wav2Vec2CTCTokenizer
11
12 tokenizer = Wav2Vec2CTCTokenizer("vocab.json", unk_token="[UNK]", pad_token="[PAD]", word_delimiter_token="|")
13
14 # Probar el tokenizer con un ejemplo
15 texto_ejemplo = "hola mundo"
16 texto_ejemplo_procesado = texto_ejemplo.replace(" ", "|")
17 tokens = tokenizer(texto_ejemplo_procesado)
18 print("Tokens generados:", tokens)
19 decoded_text = tokenizer.decode(tokens['input_ids'])
20 print("Texto decodificado:", decoded_text)
21
22
23

```

 Vocabulario creado y guardado como vocab.json. Total de caracteres: 30
 Tokens generados: {'input_ids': [28, 28, 28, 28, 0, 28, 28, 28, 28, 28], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]}
 /usr/local/lib/python3.10/dist-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean\_up\_tokenization\_spaces` was not set. It will be s
 warnings.warn(
 Texto decodificado: [UNK] [UNK]

```

1 from transformers import Wav2Vec2FeatureExtractor
2 #Define el pipeline de extraccion de caracteristicas
3 feature_extractor = Wav2Vec2FeatureExtractor(
4     feature_size=1,
5     sampling_rate=16000,
6     padding=True,
7     do_normalize=True,
8     return_attention_mask=True
9 )
10
11
12

```

```

1 from transformers import Wav2Vec2Processor
2
3 # Cargar el procesador combinando tokenizer y feature extractor
4 processor = Wav2Vec2Processor.from_pretrained("facebook/wav2vec2-base-960h")
5
6 def pipeline_procesamiento(example):
7
8     # Procesar el audio (extraer características)
9     audio_input = processor(example["audio"] ["array"],
10                             sampling_rate=example["audio"] ["sampling_rate"],
11                             return_tensors="pt",
12                             padding=True)
13
14     # Verificar si la máscara de atención está presente, ya que no siempre se genera

```

```

15     input_values = audio_input.input_values[0]
16     attention_mask = audio_input.attention_mask[0] if "attention_mask" in audio_input else None
17
18     # Procesar el texto (tokenización)
19     labels = processor.tokenizer(example["text"], return_tensors="pt", padding=True).input_ids
20
21     # Retornar los valores de entrada (input_values), la máscara de atención y las etiquetas (labels)
22     return {
23         "input_values": input_values, # Extraer características del audio
24         "attention_mask": attention_mask, # Verificar que exista la máscara de atención
25         "labels": labels[0] # IDs del texto tokenizado
26     }
27
28 # Probar con un ejemplo del dataset
29 ejemplo_procesado = pipeline_procesamiento(dataset_filtro_train[0])
30
31 # Verificar los resultados
32 print("Valores de entrada (input_values):", ejemplo_procesado["input_values"])
33 print("Máscara de atención:", ejemplo_procesado["attention_mask"]) # Puede ser None si no se genera
34 print("Etiquetas (labels):", ejemplo_procesado["labels"])
35
36
37
38

```

```

🔍 Valores de entrada (input_values): tensor([[-0.3012, -0.2587, -0.2879, ..., 0.1243, 0.0010, -0.4297]])
Máscara de atención: None
Etiquetas (labels): tensor([19, 11, 7, 23, 6, 5, 13, 4, 8, 9, 5, 4, 17, 10, 12, 12, 16, 12,
4, 13, 7, 19, 11, 5, 15, 4, 15, 22, 9, 14, 5, 4, 10, 12, 4, 12,
16, 13, 23, 13, 10, 12, 5, 14, 4, 17, 10, 12, 12, 16, 12, 4, 13, 7,
19, 11, 5, 15, 4, 15, 22, 9, 14, 5, 4, 15, 10, 25, 5, 14, 4, 29,
16, 12, 6, 4, 18, 11, 5, 13, 5, 4, 6, 11, 5, 4, 7, 25, 8, 9,
15, 5, 7, 4, 17, 7, 10, 9, 4, 13, 8, 7, 14, 4, 14, 10, 23, 23,
5, 14, 4, 14, 8, 18, 9, 4, 10, 9, 6, 8, 4, 7, 4, 15, 10, 6,
6, 15, 5, 4, 11, 8, 15, 15, 8, 18, 4, 20, 13, 10, 9, 21, 5, 14,
4, 18, 10, 6, 11, 4, 7, 15, 14, 5, 13, 12, 4, 7, 9, 14, 4, 15,
7, 14, 10, 5, 12, 4, 5, 7, 13, 14, 13, 8, 23, 12, 4, 7, 9, 14,
4, 6, 13, 7, 25, 5, 13, 12, 5, 14, 4, 24, 22, 4, 7, 4, 24, 13,
8, 8, 26])

```

```

1 #Antes de empezar, evalúe que todo está bien: escuche un par de audios y vea si el texto de referencia es correcto
2
3 from IPython.display import Audio, display
4
5 def reproducir_audios_con_textos(dataset, n=5):
6
7
8     n = min(n, len(dataset))
9
10
11     for i in range(n):
12         # Obtener el audio y el texto
13         audio_data = dataset[i]["audio"]["array"]
14         texto = dataset[i]["text"]
15
16         # Mostrar el texto de referencia
17         print(f"Texto de referencia {i+1}: {texto}")
18
19         # Reproducir el audio
20         display(Audio(audio_data, rate=dataset[i]["audio"]["sampling_rate"]))
21
22 # Reproducir y verificar los primeros 5 audios del conjunto de datos
23 reproducir_audios_con_textos(dataset_filtro_train, n=5)
24
25
26
27

```

🔍 Texto de referencia 1: CHAPTER ONE MISSUS RACHEL LYNDE IS SURPRISED MISSUS RACHEL LYNDE LIVED JUST WHERE THE AVONLEA MAIN ROAD DIPPED DOWN INTO A LITTLE HOLL

0:14 / 0:14

Texto de referencia 2: THAT HAD ITS SOURCE AWAY BACK IN THE WOODS OF THE OLD CUTHBERT PLACE IT WAS REPUTED TO BE AN INTRICATE HEADLONG BROOK IN ITS EARLIER C

0:00 / 0:15

Texto de referencia 3: FOR NOT EVEN A BROOK COULD RUN PAST MISSUS RACHEL LYNDE'S DOOR WITHOUT DUE REGARD FOR DECENCY AND DECORUM IT PROBABLY WAS CONSCIOUS TH

0:00 / 0:13

Texto de referencia 4: AND THAT IF SHE NOTICED ANYTHING ODD OR OUT OF PLACE SHE WOULD NEVER REST UNTIL SHE HAD FERRETTED OUT THE WHYS AND WHEREFORES THEREOF T

0:00 / 0:14

Texto de referencia 5: BUT MISSUS RACHEL LYNDE WAS ONE OF THOSE CAPABLE CREATURES WHO CAN MANAGE THEIR OWN CONCERNS AND THOSE OF OTHER FOLKS INTO THE BARGAIN

0:00 / 0:12

```

1 #Use la siguiente función para crear los batches en el formato que el modelo necesita
2 def prepare_dataset(batch):
3     audio = batch["audio"]
4
5     # batched output is "un-batched" to ensure mapping is correct
6     batch["input_values"] = processor(audio["array"], sampling_rate=audio["sampling_rate"]).input_values[0]
7     batch["input_length"] = len(batch["input_values"])
8
9     with processor.as_target_processor():
10         batch["labels"] = processor(batch["text"]).input_ids
11     return batch

```

```

1 from transformers import Wav2Vec2Processor
2 import torch
3 from dataclasses import dataclass
4 from typing import Any, Dict, List, Union

```

1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 26

Cambiar tipo de ento